Roteiro da demonstração ao vivo — 12/11/2026 (preparado em 02/11)

Comandos prontos em `scripts/demo_final.sh` (pausa antes de cada passo para a previsão; `scripts/demo_final.sh 3` recomeça do passo 3 se algo der errado).

N	Passo	Previsão a dizer ANTES	O que mostrar	Tempo	Quem
1	Estado inicial	"Cinco serviços saudáveis; só 8080 e 9090 local."	`docker compose ps`; apontar no diagrama	30 s	
2	Fluxo normal	"Vai alternar A, B, A, B e o id aparece nos dois logs."	6 req; `grep $rid` no Nginx e no Rust	45 s	
3	Falha de um nó	"Serviço continua; 1 a 3 req de ~1 s; DOWN em ~15 s."	`stop rust-a`; 10 req; log Nginx `504, 200`; watchdog DOWN; `/alerts`	90 s	
4	Vivo, mas não pronto	"live 200, ready 503, usuário não vê erro."	`admin.sh ready rust-b 0`; live/check; 6 req	60 s	
5	Recuperação	"A volta ao revezamento em até ~10 s após o start (DNS do Nginx), sem reinício em loop."	10 req balanceadas; `RestartCount`; alertas vazios	45 s	
6	Falha do monitor	"Usuário não percebe; MonitorParado em ~35 s. O Prometheus vigia o vigia; o Prometheus é o SPOF residual."	`stop watchdog`; idade da última sondagem; `/alerts`	30 s	

Observado no ensaio de 02/11 (`evidencias/ensaio_demo/`): todos os passos rodaram; previsões confirmadas (3 req a 1,09 s; A 5 / B 5 após recuperação; `RestartCount=0`; `MonitorParado` com idade de 44 s). `WatchdogFlapping` disparou — **corretamente** — porque o rust-a tinha passado por 3 ciclos DOWN→UP em 10 min (coleta de evidências + demo). **Esperar ≥ 10 min entre ensaio e apresentação**, ou explicar o alerta se aparecer.

Tempo total ≈ 5 min. **Atenção**: os `sleep` do script somam ~80 s; o narrador fala durante eles.

Quem faz o quê

Papel	Nome
Digita os comandos	
Narra e explica	
Cronometra e monitora logs (2º terminal: `docker compose logs -f nginx watchdog`)	
Responde perguntas	

Se algo der errado ao vivo

Narrar o que viu → formular hipótese → dizer como testaria (isso também é avaliado). Ex.: "O A não voltou ao revezamento em 10 s. Hipótese: o Nginx ainda não re-resolveu o nome. Teste: ver `could not be resolved` no error log e repetir em 5 s."

Checklist (do enunciado)

- [ ] Ambiente limpo: `docker compose down`, depois `up -d --build` e teste
- [ ] Fonte grande no terminal e janelas organizadas
- [ ] Comandos copiados num arquivo de roteiro (`scripts/demo_final.sh`) ✔
- [ ] Todos os integrantes falam
- [ ] Plano B: vídeo de tela (até 2 min) ou prints com horário
- [ ] Nenhuma porta administrativa exposta à rede (`evidencias/preliminar/09`) ✔
- [ ] Sem senhas ou tokens visíveis na tela
- [ ] Cronometrada no ensaio: cabe em 5 min